



International School

CAPSTONE PROJECT 1

CMU-SE 450

CODE STANDARD DOCUMENT

Version: 2.0

Date: 10/05/2024

DND TRANSLATE - SIGN LANGUAGE

Submitted by

Duy, Huynh Gia

Dinh, Le Nguyen Nhut

Sy, Nguyen Van

Minh, Huynh Tan Quang

Approved by

Proposal Review Panel Representative:

Name

Signature

Date

Capstone Project 2- Mentor:

Name

Signature

Date

Thuan, Nguyen Tan, MSc.

Table of Contents

1. Project Overview.....	3
2. Introduction.....	4
2.1. Purpose.....	4
2.2. Scope.....	4
3. Code Standards.....	4
3.1. Variables.....	4
3.2. Spaces Around Operators.....	4
3.3. Statement Rules.....	5
3.4. Object Rules.....	5
3.5. Line Length.....	5
3.6. Spaces.....	5
3.7. Quotes.....	5
3.8. Comparative math.....	5
3.9. Dot location.....	6
3.10. Array.....	6
3.11. Modules.....	6
3.12. Functions.....	6
3.13. String.....	6
3.14. Error catching.....	6
3.15. Files.....	6
3.16. Others.....	6

REVISION HISTORY

Version	Date	Description of Changes	Author
1.0	15/02/2024	Initial document	Team

1. Project Overview**PROJECT INFORMATION**

Project acronym	DTSL		
Project Title	DnD Translate - Sign Language		
Start Date	15 - Feb - 2024	End Date	20 - May - 2024
Lead Institution	International School, Duy Tan University		
Project Mentor	Name: Thuan, Nguyen Tan Ph.D. Email: thuannt.it.dtu@gmail.com Tel: 0905.626.276		
Scrum Master	Name: Duy, Huynh Gia Email: huynhduy25072002@gmail.com Tel: 0762.555.759		
Product Owner	Name: Dinh, Le Nguyen Nhut Email: lenguyennhutdinh@gmail.com Tel: 0965.925.991		
Team members	Name	Email	Tel
	Minh, Huynh Tan Quang	liverpool990minh12@gmail.com	0929.097.985
	Sy, Nguyen Van	vansy01682937260@gmail.com	0377.445.085

2. Introduction

2.1. Purpose

This Coding Standard requires certain practices for developing programs in the JavaScript language. The objective of this coding standard is to have a positive effect on

- Avoidance of errors/bugs, especially the hard-to-find ones.
- Maintainability, by promoting some proven design principles

2.2. Scope

- This standard pertains to the use of the JavaScript language.

3. Code Standards

3.1. Variables

- Using **camelCase** for identifier names (variables and functions).
- All names start with a **letter**.
- Constants (like PI) written in **UPPERCASE**
- No unused variables.
- For var declarations, write each declaration in its own statement.
- Avoid modifying variables of class declarations.
- Avoid modifying variables declared using const.
- No re-declaring variables.
- Avoid assigning a variable to itself.
- Avoid comparing a variable to itself.
- Restricted names should not be shadowed.

3.2. Spaces Around Operators

- Always put spaces around operators (= + - * /), and after commas.

3.3. Statement Rules

- Put the opening bracket at the end of the first line.
- Use one space before the opening bracket
- Put the closing bracket on a new line, without leading spaces.
- Keep “else” statements on the same line as their curly braces.

3.4. Object Rules

- Place the opening bracket on the same line as the object name.
- Use colon plus one space between each property and its value
- Do not add a comma after the last property-value pair.
- Place the closing bracket on a new line, without leading spaces.
- Maintain consistency of newlines between object properties.
- Always end an object definition with a semicolon.

3.5. Line Length

- For readability, avoid lines longer than 80 characters

3.6. Spaces

- Use 2 spaces for indentation.
- Add a space after keywords.
- Add a space before a function declaration's parentheses
- Commas should have a space after them.
- Add spaces inside single line blocks.
- No space between function identifiers and their invocations.
- Add space between colon and value in key value pairs.

3.7. Quotes

- Use single quotes for strings except to avoid escaping.

3.8. Comparative math

- Always use === instead of ==.
- Exception: obj == null is allowed to check for null || undefined.

3.9. Dot location

- Dot should be on the same line as property.

3.10. Array

- Use array literals instead of array constructors

3.11. Modules

- Use a single import statement per module.
- Renaming import, export, and destructuring assignments to the same name are not allowed.

3.12. Functions

- Avoid unnecessary function binding.
- No unnecessary parentheses around function expressions.
- No function declarations in nested blocks.

3.13. String

- Regular strings must not contain template literal placeholders.
- No octal escape sequences in string literals.
- No multiline strings.
- No spacing in template strings.

3.14. Error catching

- Only throw an Error object.

3.15. Files

- Files must end with a newline.

3.16. Others

- Semicolons must have a space after and no space before.
- Must have a space before blocks.
- Use isNaN() when checking for NaN
- Function typeof must be compared to a valid string.
- Never start a line with (, [, `, or a handful of other unlikely possibilities.